02 / ALGEBRA / CLASS_17_VECTORS / EX26

Pseudocode guide

Turn mathematical formulas and conditions into programming steps.

DOCUMENT 04

Learning objectives

- Turn formulas into an understandable algorithm.
- Identify data, operations, decisions and checks.
- Write your own pseudocode before coding.

Prerequisites: Prerequisites: EX25 completed, NumPy arrays, vector components and Euclidean norm.

Reading order: 01 Theory → 02 Reference → 03 Exercise → 04 Pseudocode → Coding.

1. From math problem to algorithm

Pseudocode is not Python: it describes logical steps without worrying about exact syntax. Its purpose is to explain the problem before coding it.

> Course rule: mathematical concepts → algebraic example → physical application → algorithm → code → verification.

2. How to translate a formula

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y$$

A formula contains inputs (a, b), operations (multiply and add), and one output (a scalar). The algorithm must make those three parts explicit.

```
START
  READ vector a and vector b
  CHECK that dimensions match
  COMPUTE the dot product of a and b
  DISPLAY the result
END
```

3. Guided pseudocode for EX26-A

Use this outline to create your own full pseudocode; some steps are deliberately left open for you.

```
START BLOCK A
  DEFINE the input vectors
  COMPUTE the dot product
  COMPUTE both norms
  COMPUTE cos(theta) from the theory formula
  [THINK: which angle conversion is needed?]
  DISPLAY the results
  COMPARE with check values
END BLOCK A
```

4. Independent pseudocode for EX26-B

For the physics problem, write the outline yourself instead of copying Part A. It must include at least:

- Inputs F and s with their units.
- Mathematical operation needed for W.
- Angle calculation and norm check.
- Readable output in joules and degrees.
- Repetition for perpendicular and opposing cases.

5. From pseudocode to Python

Match each logical verb with a programming action. A logical verb is not necessarily one specific Python function: several implementations may work.

DEFINE → `np.array([...])`

COMPUTE dot → `np.dot(...)`

COMPUTE norm → `np.linalg.norm(...)`

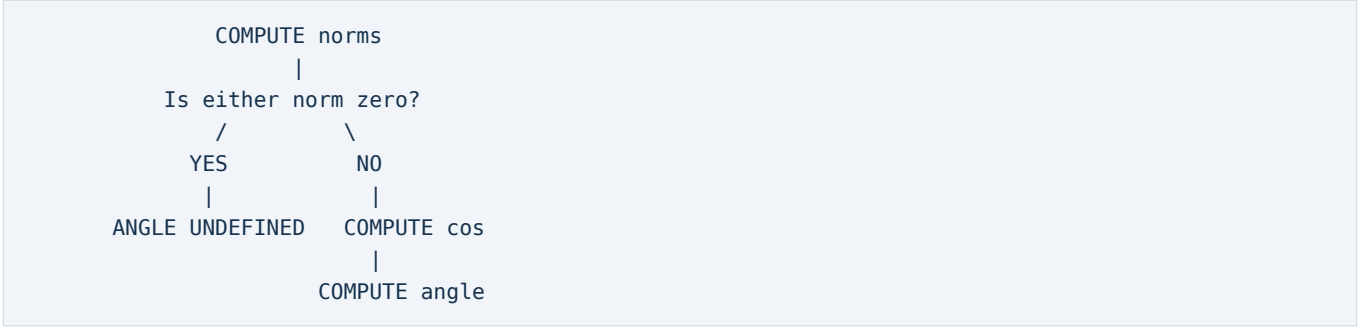
GET angle → `np.arccos(...)`

CONVERT → `np.degrees(...)`

DISPLAY → `print(...)`

6. When do decisions appear?

Although EX26 inputs avoid zero vectors, mathematically an angle requires both norms to be nonzero. This illustrates a decision that we will later implement using if and then error handling.



This diagram is a preview. When we study iterative algorithms, we will use full flowcharts with input, process, decision shapes and arrows.

7. Checklist before coding

- Are all input vectors and dimensions defined?
- Do I know which operations return numbers and which return vectors?
- Have I checked the requirements for the angle calculation?
- Have I included work units in the result?
- Do I have independent check values?